

Shared Responsibility Model

Task 1

Create a comparison table showing which security responsibilities belong to the cloud provider and which belong to the user for a food delivery app like Zomato running on AWS, Azure, or Google Cloud.

Cloud Provider's Responsibility	User's (Zomato's) Responsibility
Physical security of data centres, servers, and hardware.	Securing the application code (e.g., the Zomato app and website).
Maintaining the underlying network infrastructure and host operating system.	Configuring firewalls, security groups, and network access rules correctly.
Ensuring the availability and durability of core services such as compute, storage, and databases.	Managing IAM users, roles, and permissions for employees and services.
Patching and updating the virtualization layer and physical infrastructure.	Encrypting customer data (order details, payment info, addresses) at rest and in transit.
Providing built-in compliance certifications for the infrastructure (e.g., ISO, SOC).	Setting correct access permissions on storage buckets/databases holding restaurant and customer data.
Offering security tools and services (e.g., IAM, monitoring, encryption options) for customers to use.	Regularly monitoring, logging, and auditing configurations to catch mistakes early.

Task 2

Sign up for a free trial on AWS, Azure, or Google Cloud, and create a new user account with limited permissions (for example, only allow access to storage buckets or databases, but not to billing or admin settings).

Steps followed:

1. Logged in to the AWS Management Console using the root account created earlier.
2. Opened the IAM (Identity and Access Management) service from the search bar.
3. Selected "Users" from the left-hand menu and clicked "Create user".
4. Entered a username (e.g., "storage-limited-user") and enabled console access with an auto-generated password.
5. On the permissions step, chose "Attach policies directly" instead of adding the user to an admin group.
6. Selected only storage-related policies (restricted to S3), and made sure that policies like Billing, IAMFullAccess, and AdministratorAccess were not attached.
7. Reviewed the summary to confirm the new user had access only to storage and no access to billing or admin settings, then clicked "Create user".

8. Tested the setup by logging in as the new user and confirming that the Billing and IAM sections showed "Access Denied", while the storage section opened normally.

Task 3

Configure a custom policy in your chosen cloud platform that allows a user to upload images to a storage bucket but denies them permission to delete any files.

Steps followed:

1. In the IAM console, went to "Policies" and clicked "Create policy".
2. Switched to the JSON editor tab to write a custom policy instead of using the visual editor.
3. Added a statement that allows the "PutObject" action (which permits uploading files) on the target storage bucket.
4. Added a second statement that explicitly denies the "DeleteObject" action on the same bucket, so the user can never delete files even by mistake.
5. Reviewed the policy to confirm PutObject was set to "Allow" and DeleteObject was set to "Deny", then named it (e.g., "UploadOnly-NoDelete-Policy") and created it.
6. Attached this custom policy to the limited user created in Task 2.
7. Verified the setup by logging in as that user: uploading a test image to the bucket succeeded, while attempting to delete any file from the bucket returned an "Access Denied" error, confirming the policy worked as intended.

Task 4

Simulate a real-world scenario: Imagine you are part of a team managing a cricket ticket booking app (like BookMyShow). Assign the following roles using your cloud provider's IAM or equivalent: one user can only view booking data, another can edit booking data, and a third can manage user accounts.

Steps followed:

1. Opened the IAM console and created three separate users: "booking-viewer", "booking-editor", and "account-manager".
2. For "booking-viewer", attached a read-only policy scoped to the booking data storage/database, so this user can only view records.
3. For "booking-editor", created a custom policy that allows view and update actions on the booking data, so this user can view and edit bookings but still cannot delete the storage bucket or change other users' permissions.
4. For "account-manager", attached a custom IAM-management policy that allows creating, updating, and removing other user accounts, but with no access at all to the booking data itself.

5. Verified each role separately by logging in as each user and confirming that permissions matched exactly what was assigned — for example, "booking-viewer" could not edit or delete anything, and "account-manager" could not open the booking database.

Roles summary:

User	Access Given	Purpose
booking-viewer	Read-only access to booking data	Can only view booking records, cannot edit or delete anything
booking-editor	Read and write access to booking data	Can view and update bookings, cannot manage users or delete resources
account-manager	IAM/user management permissions	Can create, update, and remove user accounts, no access to booking data

Task 5

Find and describe one real security incident from the last 3 years where a cloud misconfiguration (wrong permissions or policies) exposed user data. Explain in 3-4 lines how applying the Shared Responsibility Model could have prevented it.

Incident: Toyota cloud misconfiguration (2023)

In May 2023, Toyota disclosed that a misconfigured cloud database exposed the location data and personal information of about 2.15 million customers in Japan for almost ten years, from 2013 to 2023. Soon after, Toyota found additional misconfigured cloud services that had leaked further customer data, including names, addresses, phone numbers, and vehicle identification numbers, for several more years. The root cause was that the database had been left publicly accessible due to incorrect access settings, rather than any flaw in the cloud provider's infrastructure itself.

How the Shared Responsibility Model could have prevented it:

Under the shared responsibility model, the cloud provider is responsible for securing the underlying infrastructure, but Toyota, as the customer, was responsible for correctly configuring access permissions on its own databases and storage. If Toyota had properly restricted the database to private access, enforced the principle of least privilege on its IAM roles, and run regular audits or automated configuration checks, this public exposure could have been caught and fixed within days instead of persisting for nearly a decade.